ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO THỰC HÀNH
VI XỬ LÝ – VI ĐIỀU KHIỂN

LAB 3: NÚT NHẤN VÀ CÔNG TẮC

(Buttons/Switches – đèn giao thông ngã tư)

Lớp: **L01** – HK261
Giảng viên hướng dẫn: **Lê Trọng Nhân**
Sinh viên thực hiện: **Đào Ngọc Thanh Tú** – **2413841**

Link source code (GitHub)

`https://github.com/thanhtu988/bao-cao-vi-xu-ly-lab3`

Thành phố Hồ Chí Minh – Ngày 11 tháng 11 năm 2026

I. MÔI TRƯỜNG VÀ CẤU HÌNH

1. Công cụ sử dụng

Vi điều khiển	STM32F103C6Ux (Cortex-M3, HSI 8 MHz)
IDE / Sinh mã	STM32CubeIDE 1.7.0 + CubeMX 6.3.0, HAL FW_F1 V1.8.7
Mô phỏng	Proteus 8.10 SP0
Mã nguồn	1 project CubeIDE `Lab3` (tích hợp cả 10 exercise)

2. Phân bố chân và mức logic

Chân	**Nhãn**	**Mức bật**	**Chức năng**
PA1–PA6	N_*, S_* (RED/AMBER/GREEN)	0	6 đèn tuyến 1 (Bắc, Nam)
PA7–PA12	E_*, W_* (RED/AMBER/GREEN)	0	6 đèn tuyến 2 (Đông, Tây)
PB0–PB6	SEG0–SEG6	0	bus a–g chung, LED 7 đoạn chung anode
PB8–PB11	EN0–EN3	0	EN0–1: tuyến 1, EN2–3: tuyến 2 (qua PNP)
PB12–PB14	BUTTON_1–3	0 = nhấn	Input, điện trở 10K kéo lên 3.3V (Figure 1.3)

3. Tổ chức file (Figure 1.6)

`input_reading.c/h`	đọc 3 nút mỗi 10 ms, chống dội (Program 1.2)
`input_processing.c/h`	FSM nút nhấn, FSM chế độ, FSM đèn giao thông
`led_display.c/h`	`led_buffer`, quét LED 7 đoạn, bật/tắt 12 đèn
`timer.c/h`	`TIMER_CYCLE`, software timer, callback ngắt TIM2
`main.c`	chỉ gọi `fsm_for_input_processing()` trong `while(1)` (Program 1.9)

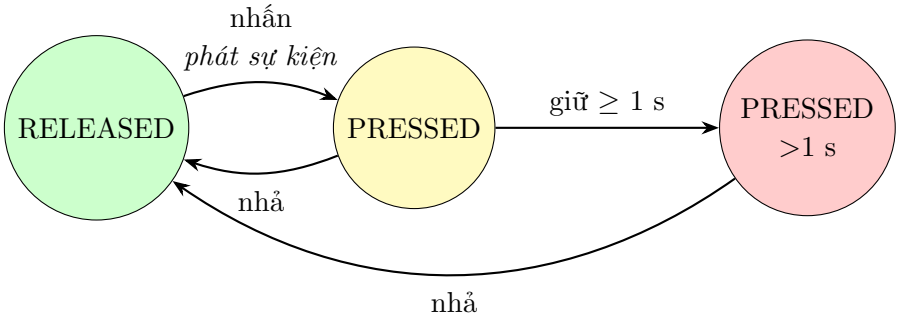
II. KẾT QUẢ 10 EXERCISE

Exercise 1 – Sketch an FSM

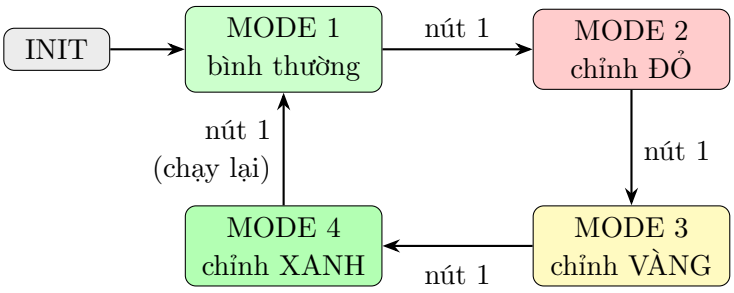
Yêu cầu: vẽ FSM mô tả cách giải bài toán đèn giao thông có 3 nút nhấn.

Bài toán dùng **ba FSM** chạy song song trong `fsm_for_input_processing()`.

(a) FSM nút nhấn (Figure 1.9, dùng chung cho 3 nút) – sự kiện "nhấn" chỉ phát ra **một lần**.

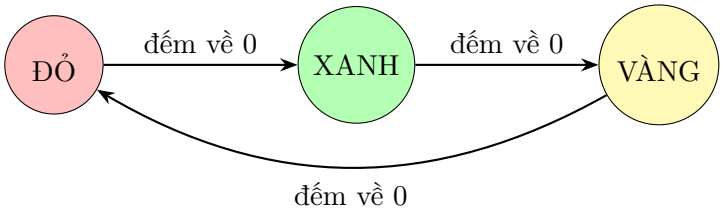


(b) FSM chế độ – nút 1 chuyển mode theo vòng $1 \to 2 \to 3 \to 4 \to 1$.



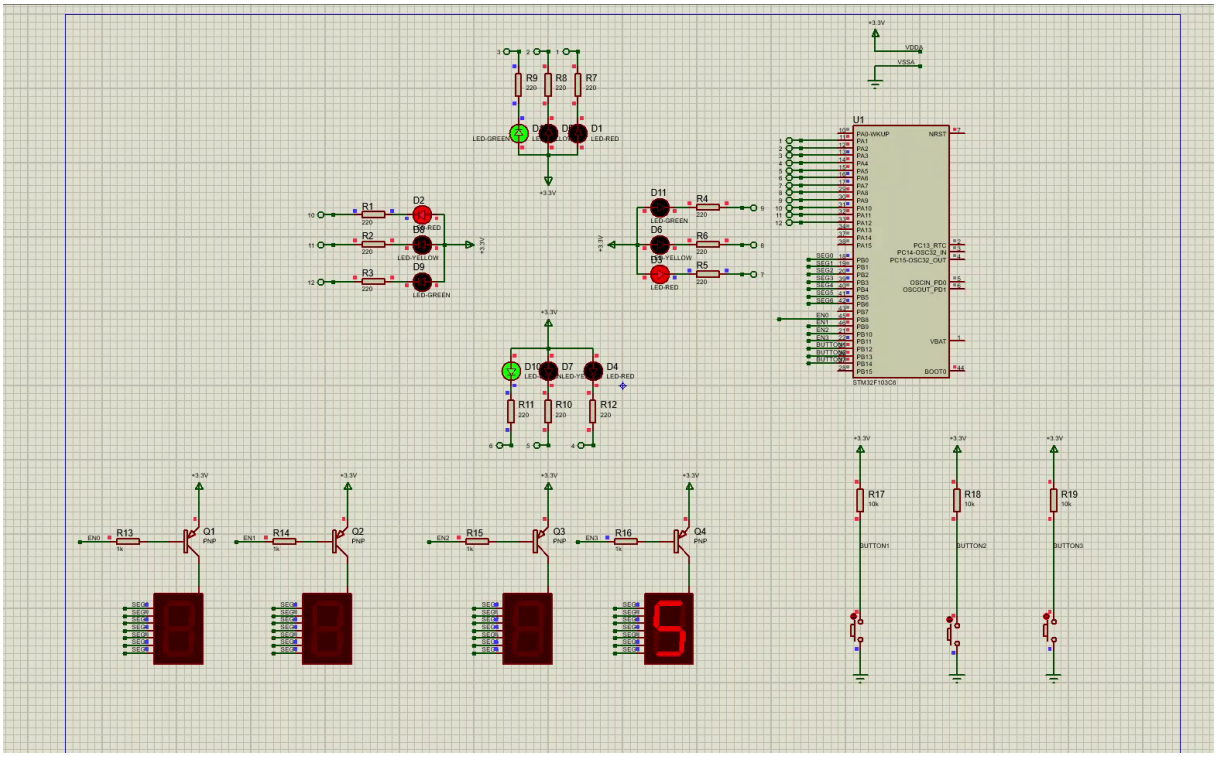
Ở mode 2, 3, 4: 4 đèn cùng màu nhấp nháy 2 Hz; nút 2 tăng giá trị tạm ($99 \to 1$); nút 3 lưu; tuyến 1 hiện giá trị, tuyến 2 hiện số mode.

(c) FSM đèn mỗi tuyến (mode 1): ĐỎ → XANH → VÀNG → ĐỎ, đổi màu khi bộ đếm giây về 0. Tuyến 1 bắt đầu ĐỎ, tuyến 2 bắt đầu XANH.



Exercise 2 – Proteus Schematic

Yêu cầu: vẽ sơ đồ Proteus cho bài toán.



Hình 1: Exercise 2 – sơ đồ đèn giao thông ngã tư trong Proteus

Gồm: STM32F103C6, 12 LED (4 đỏ, 4 vàng, 4 xanh, anode lên 3.3V qua 220Ω), 4 LED 7 đoạn chung anode (bus SEG chung, chân chung qua PNP), 3 nút nhấn có điện trở kéo lên 10K.

Exercise 3 – Create STM32 Project

Yêu cầu: tạo project có chân đúng theo sơ đồ, ngắt timer khoảng 10 ms.

GPIO Output: Push Pull, mức ban đầu **High** (đèn tắt). GPIO Input PB12–PB14: No pull. TIM2: Internal Clock, PSC = 7999, ARR = 9, bật NVIC.

$$f_{\text{ngắt}} = \frac{8\,000\,000}{(7999+1)\times(9+1)} = 100 \text{ Hz} \;\Rightarrow\; T_{\text{ngắt}} = 10 \text{ ms}$$

```
MX_GPIO_Init();
MX_TIM2_Init();
HAL_TIM_Base_Start_IT(&htim2);
while (1)
{
  fsm_for_input_processing();
}
```

Listing 1: Exercise 3 – main.c (Program 1.9)

Exercise 4 – Modify Timer Parameters

Yêu cầu: khi đổi chu kì ngắt (1 ms, 100 ms) thì sửa ít nhất và không ảnh hưởng hệ thống (đèn vẫn 2 Hz).

Giải pháp: mọi thời gian đều viết bằng **ms**, chỉ đổi ra số nhịp ngắt ở một chỗ là `setTimerX()` (chia cho `TIMER_CYCLE`). Đổi chu kì ngắt chỉ cần sửa PSC/ARR và `TIMER_CYCLE`.

```
#define TIMER_CYCLE   10

void setTimer0(int duration)
{
  timer0_counter = duration / TIMER_CYCLE;
  timer0_flag    = 0;
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  if (htim->Instance == TIM2) {
    button_reading();
    timer_run();
  }
}
```

Listing 2: Exercise 4 – timer.h, timer.c

```
#define ONE_SECOND          1000
#define BLINK_DURATION      250
#define SCAN_DURATION       (1000 / MAX_LED)
```

Listing 3: Exercise 4 – cac thoi gian viet bang ms (input_processing.c)

Chu kì ngắt	PSC	ARR	TIMER_CYCLE	Đèn 2 Hz (250 ms)
1 ms	799	9	1	250 nhịp – đúng
10 ms	7999	9	10	25 nhịp – đúng
100 ms	7999	99	100	2 nhịp = 200 ms

2 Hz = một chu kì sáng–tắt 0,5 s ⇒ đảo trạng thái mỗi **250 ms**. Ở 100 ms, `250/100` chia nguyên còn 2 nhịp nên đèn đảo mỗi 200 ms: software timer không đo được khoảng lẻ hơn chu kì ngắt. Với 1 ms phải dùng PSC = 799, ARR = 9 (không dùng ARR = 0 như đã gặp ở Lab 2).

Exercise 5 – Adding code for button debouncing

Yêu cầu: thêm file đọc input và hiển thị output, chống dội nút, nhấn nút 1 thì tăng mode.

Chống dội theo Program 1.2: mỗi 10 ms đọc chân một lần, chỉ cập nhật `buttonBuffer` khi **hai lần đọc liên tiếp giống nhau**. Mở rộng cho 3 nút bằng mảng `buttonPort[]`, `buttonPin[]`.

```
void button_reading(void)
{
  for (uint8_t i = 0; i < NO_OF_BUTTONS; i++) {
    debounceButtonBuffer2[i] = debounceButtonBuffer1[i];
    debounceButtonBuffer1[i] = HAL_GPIO_ReadPin(buttonPort[i],
       buttonPin[i]);
    if (debounceButtonBuffer1[i] == debounceButtonBuffer2[i])
      buttonBuffer[i] = debounceButtonBuffer1[i];
    if (buttonBuffer[i] == BUTTON_IS_PRESSED) {
      if (counterForButtonPress1s[i] <
         DURATION_FOR_AUTO_INCREASING) {
        counterForButtonPress1s[i]++;
      } else {
        flagForButtonPress1s[i] = 1;
      }
    } else {
      counterForButtonPress1s[i] = 0;
      flagForButtonPress1s[i] = 0;
    }
  }
}
```

Listing 4: Exercise 5 – input_reading.c

Xử lý nút theo FSM Program 1.7; thay "tăng PORTA" bằng trả về 1 để báo nút vừa được nhấn:

```
static unsigned char fsm_for_button(unsigned char index)
{
  unsigned char pressed = 0;

  switch (buttonState[index]) {
  case BUTTON_RELEASED:
    if (is_button_pressed(index)) {
      buttonState[index] = BUTTON_PRESSED;
      pressed = 1;
    }
    break;
  case BUTTON_PRESSED:
    if (!is_button_pressed(index)) {
      buttonState[index] = BUTTON_RELEASED;
    } else {
      if (is_button_pressed_1s(index)) {
        buttonState[index] = BUTTON_PRESSED_MORE_THAN_1_SECOND;
      }
    }
```

```
    break;
  case BUTTON_PRESSED_MORE_THAN_1_SECOND:
    if (!is_button_pressed(index)) {
      buttonState[index] = BUTTON_RELEASED;
    }
    break;
  }
  return pressed;
}
```

Listing 5: Exercise 5 – input_processing.c: FSM nut nhan

Exercise 6 – Adding code for displaying modes

Yêu cầu: hiện mode trên LED 7 đoạn và nhấp nháy đèn theo mode.

Vào mode 2–4: tắt 12 đèn, tuyến 1 hiện giá trị đang chỉnh, tuyến 2 hiện số mode, timer1 đảo 4 đèn cùng màu mỗi 250 ms.

```
static void enter_modify_mode(int current_value, int mode_number)
{
  clearAllTrafficLights();
  temp_value = current_value;
  update7SEGBuffer(temp_value, mode_number);
  setTimer1(BLINK_DURATION);
}

void toggleTrafficColor(int color)
{
  HAL_GPIO_TogglePin(N_RED_GPIO_Port, ROAD_PIN[ROAD1][color] |
     ROAD_PIN[ROAD2][color]);
}
```

Listing 6: Exercise 6 – vao mode chinh sua va dao den

```
void update7SEG(int index)
{
  HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin | EN1_Pin | EN2_Pin |
     EN3_Pin, EN_OFF);

  switch (index) {
  case 0:
    display7SEG(led_buffer[0]);
    HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, EN_ON);
    break;
  case 1:
    display7SEG(led_buffer[1]);
    HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, EN_ON);
    break;
  case 2:
```

```
    display7SEG(led_buffer[2]);
    HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, EN_ON);
    break;
  case 3:
    display7SEG(led_buffer[3]);
    HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, EN_ON);
    break;
  default:
    break;
  }
}
```

Listing 7: Exercise 6 – quet LED 7 doan (led_display.c)

Tần số quét: timer2 gọi `update7SEG()` mỗi `SCAN_DURATION` = 1000/4 = 250 ms, giống Lab 2. Một vòng 4 LED là $T = 4 \times 0{,}25 = 1$ s $\Rightarrow f = 1$ Hz.

Exercise 7 – Increasing time duration value for the red LEDs

Yêu cầu: nút 2 tăng thời gian đèn đỏ, nút 3 lưu giá trị.

```
static void increase_temp_value(int mode_number)
{
  temp_value++;
  if (temp_value > MAX_DURATION) temp_value = MIN_DURATION;
  update7SEGBuffer(temp_value, mode_number);
}

  case MODE_2_RED:
    if (timer1_flag == 1) {
      setTimer1(BLINK_DURATION);
      toggleTrafficColor(COLOR_RED);
    }
    if (inc_pressed) increase_temp_value(2);
    if (set_pressed) duration_red = temp_value;
    if (mode_pressed) {
      mode = MODE_3_AMBER;
      enter_modify_mode(duration_amber, 3);
    }
    break;
```

Listing 8: Exercise 7 – tang gia tri 1 - 99 va mode 2

Exercise 8, 9 – Amber và green LEDs

Yêu cầu: tương tự Exercise 7 cho đèn vàng (mode 3) và đèn xanh (mode 4). Hai mode dùng chung `increase_temp_value()`, chỉ khác biến được lưu. Ở mode 4, nút 1 quay về mode 1 và chạy lại với giá trị mới.

```
case MODE_3_AMBER:
  if (timer1_flag == 1) {
    setTimer1(BLINK_DURATION);
    toggleTrafficColor(COLOR_AMBER);
  }
  if (inc_pressed) increase_temp_value(3);
  if (set_pressed) duration_amber = temp_value;
  if (mode_pressed) {
    mode = MODE_4_GREEN;
    enter_modify_mode(duration_green, 4);
  }
  break;

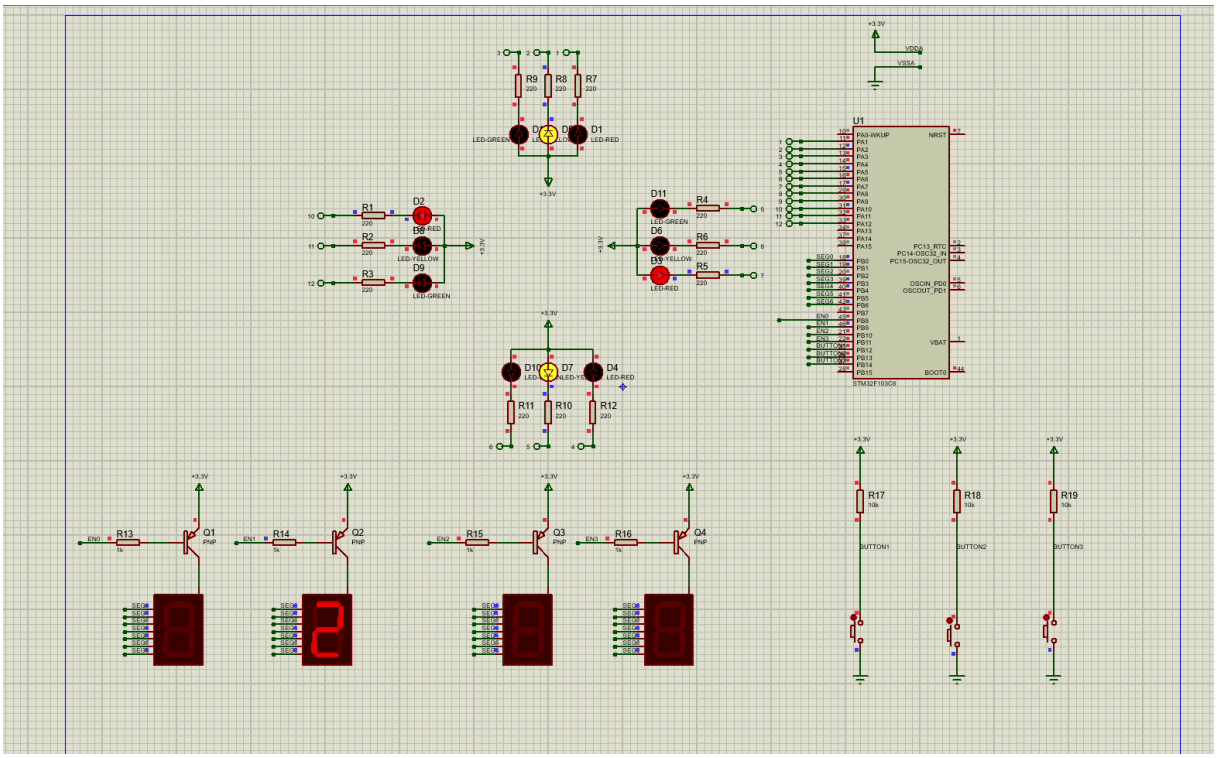
case MODE_4_GREEN:
  if (timer1_flag == 1) {
    setTimer1(BLINK_DURATION);
    toggleTrafficColor(COLOR_GREEN);
  }
  if (inc_pressed) increase_temp_value(4);
  if (set_pressed) duration_green = temp_value;
  if (mode_pressed) {
    mode = MODE_1_NORMAL;
    traffic_start();
  }
  break;
```

Listing 9: Exercise 8, 9 – mode 3 va mode 4

Exercise 10 – To finish the project

Yêu cầu: tích hợp tất cả vào một project, quay video, nộp báo cáo và code.

Tất cả nằm trong project `Lab3`. Mode 1 chạy FSM đèn của Exercise 1(c), mỗi giây giảm bộ đếm của 2 tuyến:



Hình 2: Exercise 10 – mode 1: tuyến 1 đang VÀNG (02), tuyến 2 đang ĐỎ (02)

```c
static void traffic_tick(int road)
{
  light_counter[road]--;
  if (light_counter[road] <= 0) {
    switch (light_color[road]) {
    case COLOR_RED:
      light_color[road]   = COLOR_GREEN;
      light_counter[road] = duration_green;
      break;
    case COLOR_GREEN:
      light_color[road]   = COLOR_AMBER;
      light_counter[road] = duration_amber;
      break;
    case COLOR_AMBER:
      light_color[road]   = COLOR_RED;
      light_counter[road] = duration_red;
      break;
    default:
      break;
    }
    setTrafficLight(road, light_color[road]);
  }
}
```

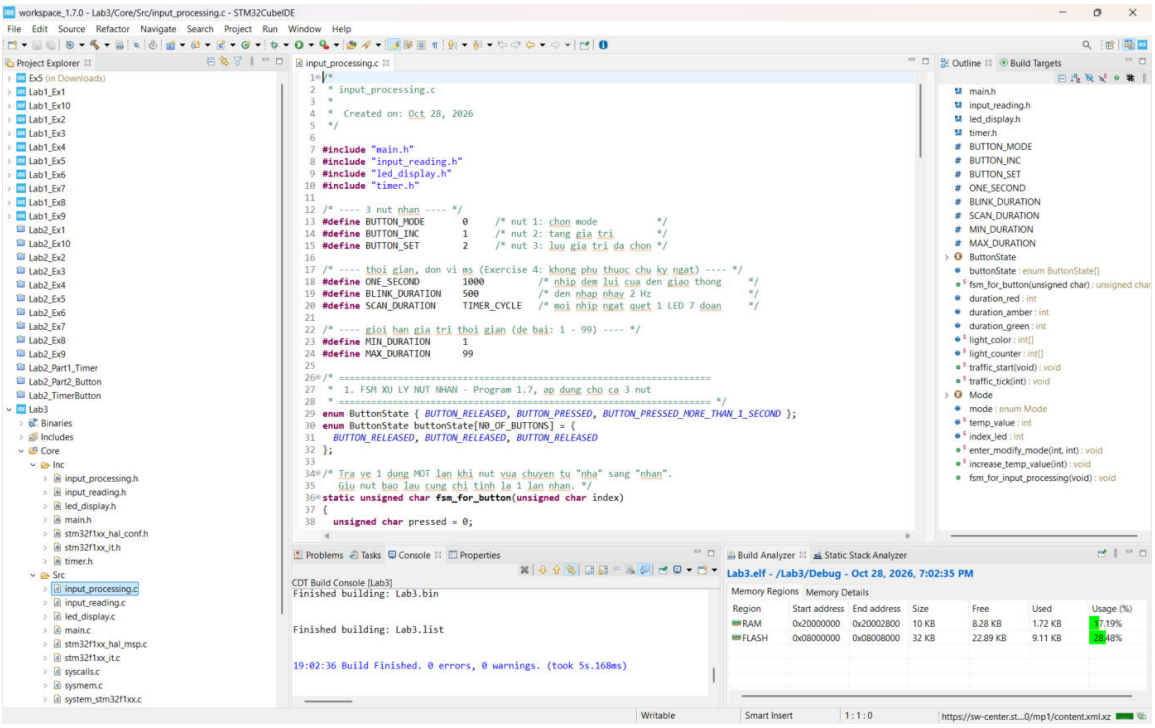
Listing 10: Exercise 10 – FSM den giao thong

Video demo: (dán link video demo vào đây)

Mặc định ĐỎ 5 s, XANH 3 s, VÀNG 2 s (lấy theo Lab 1). Để hai tuyến khớp pha cần đặt ĐỎ = XANH + VÀNG.

III. TỔNG KẾT

Ex	Nội dung	Build	Mô phỏng
1	Vẽ FSM (nút, chế độ, đèn)	–	–
2	Sơ đồ Proteus	–	✓
3	Tạo project, TIM2 10 ms	✓	✓
4	Đổi chu kì ngắt chỉ sửa 1 hằng số	✓	✓
5	Chống dội, nút 1 đổi mode	✓	✓
6	Hiện mode, nhấp nháy 2 Hz	✓	✓
7–9	Chỉnh thời gian đèn đỏ, vàng, xanh	✓	✓
10	Tích hợp 1 project, video	✓	✓



Hình 3: Build trong STM32CubeIDE – 0 errors, 0 warnings

Source code:

https://github.com/thanhtu988/bao-cao-vi-xu-ly-lab3